

Questions

1. how to access private methods outside class
2. compiler says this `animal.sound(new Integer(1))`; is deprecated and correctly working with `animal.sound((Integer) 1)`;
give proper reason with deep explanation
3. Why use Interface → Abstract Class → Child Classes pattern?

Native method in java

Is this valid?

```
class Outer {  
    private int x = 10;  
  
    class Inner {  
        void show(){  
            System.out.println(x);  
        }  
    }  
}
```

*

1 point

Yes

No

Yes if x is declared inside Inner only

Yes if x is public only

```
class Main {  
    public static void main(String[] args) {  
        try
```

```

    {
        System.out.print("Inside try ");
        throw new RuntimeException();
    }
    catch(RuntimeException e)
    {
        throw e;
    }
    catch(Exception e)
    {
        System.out.print("Inside catch ");
    }
    System.out.println("After try");
}
}
*
```

1 point

Inside try Inside catch After try
 Compilation Error
 Runtime Excpetion
 Inside try After try

```

class A {
    static void show(){
        System.out.print("A ");
    }
}

class B extends A {
    static void show(){
        System.out.print("B ");
    }
}

public class Test {
```

```

    public static void main(String[] args){
        A obj = new B();
        obj.show();
    }
}

```

*

1 point

Runtime Error

A

Compilation Error

B

Which statement is true?

*

1 point

Abstract class cannot have constructors

Abstract class cannot be instantiated

Abstract class cannot contain concrete methods

Abstract class cannot contain static methods

```

class Test {

```

```

    void m(Object o){
        System.out.print("Object ");
    }

```

```

    void m(String s){
        System.out.print("String ");
    }

```

```

    public static void main(String[] args){
        Test t = new Test();
        t.m(null);
    }

```

```
}  
*  
1 point  
Object  
String  
Error  
Exception
```

Which is true?

```
*  
1 point  
Static inner class cannot be instantiated  
Static inner class cannot access instance members directly  
Static inner class must be abstract  
Static inner class cannot access static members
```

```
ExecutorService service = Executors.newCachedThreadPool();
```

```
*  
1 point  
Maximum two threads  
Fixed number of threads  
Only one thread allowed  
Creates new threads as needed
```

```
Thread t = new Thread();  
t.start();  
t.start();  
*  
1 point  
Deadlock  
IllegalThreadStateException  
Runs twice  
Compilation error
```

```

class Test implements A,B {
    public static void main(String[] args){
        new Test().show();
        System.out.print("C");
    }
}

interface A {
    default void show(){ System.out.print("A"); }
}

interface B {
    default void show(){ System.out.print("B"); }
}

```

*

1 point

AC

BC

B

Error

```

public class Test {
    public static void main(String[] args) {
        A obj = new B();
        B b = obj;
        System.out.println("done");
    }
}

```

```

class A {}
class B extends A {}

```

*

1 point

done

Error
None
Exception

```
ExecutorService service = Executors.newFixedThreadPool(2);
```

```
for(int i=1;i<=5;i++){  
    int num = i;  
    service.submit(() -> {  
        System.out.print(num + " ");  
        try{ Thread.sleep(1000); }catch(Exception e){}  
    });  
}
```

```
service.shutdown();
```

*

1 point

All 5 tasks run simultaneously

Only 2 tasks run at a time

Tasks run sequentially

Program throws exception

If we have to use `this()` and `super()` in a sub class construct or the what must be the order??

*

1 point

`super()` then `this()`

order does not matter

can not use both at the same time

`this()` then `super()`

Which statement about `RuntimeException` and checked exceptions is TRUE?

*

1 point

You can declare a checked exception in a `throws` clause even if the method body never `throws` it, but you cannot `do` the same `with` unchecked exceptions

A method that calls a method throwing a checked exception must either declare it `with throws` or handle it – no exceptions

`NullPointerException` is a checked exception because it is very common and developers should always handle it

`Overriding` methods can declare broader checked exceptions than the parent method

```
class A {  
    void m() throws Exception { }  
}
```

```
class B extends A {  
    void m() throws IOException { }  
}
```

*

1 point

`Runtime` exception

Compiles Successfully

Must declare `Exception`

Compilation Error

Which statement about thread priority in `Java` is **MOST** accurate?

*

1 point

Setting a thread's priority above `Thread.MAX_PRIORITY` silently caps it at `10` without any exception

`Thread` scheduling based on priority is platform-dependent and the `JVM` merely `provides` a hint `to` the `OS` scheduler

`Thread` priorities are absolute guarantees enforced by the `JVM`

across all platforms

A thread with priority 10 is guaranteed to always execute before a thread with priority 1

```
interface Test {  
    void show();  
}  
  
public class Main {  
    public static void main(String[] args) {  
  
        Test t = () -> {};  
        t.show();  
    }  
}
```

```
interface Animal{
    void sound();
    void eat();
}

abstract class AbstractAnimal implements Animal{
    public void sound(){
        sout("sound");
    }
}

class Cat extends AbstractAnimal{
    public void eat(){
        sout("eat");
    }
}

class Dog extends AbstractAnimal{
    public void eat(){
        sout("eat");
    }
}

public class Main{
    public static void main(String[] args){
        Animal Dog = new Dog();
        Animal cat = new Cat();

        dog.sound();
        cat.sound();
        cat.eat();
    }
}
```

```

class Test {
    final int[] arr = {1, 2, 3};

    void change() {
        arr[0] = 99;
    }

    public static void main(String[] args) {
        new Test().change();
    }
}

```

Compilation Error

Runtime Exception

Compiles in java 8 +

Works fine

```

class Demo {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            System.out.println("Child Thread");
        });

        t.run();
        System.out.println("Main Thread");
    }
}

```

```

class A {}
class B extends A {}
class C extends B {}

public class Test {
    public static void main(String[] args) {
        A a = new C();
        B b = (B) a;
    }
}

```

```

        C c = (C) b;
        System.out.print("OK");
    }
}

```

Compilation Error

Prints OK

Runtime Exception

Undefined Behaviour

```

class Test {
    Test ref;

    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = new Test();

        t1.ref = t2;
        t2.ref = t1;

        t1 = null;
        System.gc();
    }
}

```

Both are eligible for garbage collection

only t1 is eligible for garbage collection

only t2 is eligible for garbage collection

Both are not eligible for garbage collection

```

class Test {
    static int x = 10;
    static { x = 20; }
    static { x = 30; }
    public static void main(String[] args) {
        System.out.print(x);
    }
}

```

```
}  
}  
  
10  
20  
30  
Error
```

Find the issue:

```
class Person {  
    String name;  
    public boolean equals(Person p) {  
        return this.name.equals(p.name);  
    }  
}
```

No issue

`equals()` is not properly overridden

Should return void

Should be static

How many objects are created?

```
String s1 = "Hello";  
String s2 = "Hello";  
String s3 = new String("Hello");
```

```
String s4=s2;
```

1
2
3
4

```
Integer x = null;  
int y = x;  
System.out.print(y);
```

0

null

Compilation Error

Runtime Exception

What happens when this code executes?

```
class CustomException extends Exception { }  
public class Test {  
    public static void main(String[] args) {  
        throw new CustomException();  
    }  
}
```

Compiles and runs successfully

Compilation error: unreported exception

Runtime exception

No error, exception is silently ignored

What will be the output of the following code?

```
class Animal {  
    private void sound() { System.out.print("Animal "); }  
    public void makeSound() { sound(); }  
}  
class Dog extends Animal {  
    private void sound() { System.out.print("Bark "); }  
    public void makeSound() { sound(); }  
}  
public class Test {
```

```

    public static void main(String[] args) {
        Animal a = new Dog();
        a.makeSound();
    }
}

```

Bark
 Animal
 Compilation Error
 Runtime Exception

Find the Error

```

class Person implements Serializable {
    String name;
    transient int age;
    static final long serialVersionUID = 1L;
}
class Employee extends Person {
    double salary;
}

```

No Error
 Employee must implement Serializable
 Cannot extend serializable class
 transient cannot be used with int

What is the output?

```

public class Test {
    static int method() {
        try {
            return 1;
        } catch (Exception e) {
            return 2;
        } finally {

```

```

        return 3;
    }
}
public static void main(String[] args) {
    System.out.print(method());
}
}
1
2
3
Compilation Error

```

```

enum Size {
    S(10), M(20), L(30);
    private int value;
    Size(int v) { value = v; }
    public int getValue() { return value; }
}
public class Test {
    public static void main(String[] args) {
        System.out.print(Size.M.getValue());
    }
}

10
20
30
Error

```

```

abstract class A {
    static void m() {
        System.out.print("A ");
    }
}

```

```
class B extends A {}

public class Test {
    public static void main(String[] args) {
        A a = new B();
        a.m();
    }
}
```

A
B
Compilation Error
Runtime Error

```
Object o = new String("Hello");
String s = o;
```

Compilation error
Runtime error
No error
NullPointerException

What is output?

```
interface A {
    default void show() { System.out.print("A "); }
}
interface B {
    default void show() { System.out.print("B "); }
}
class C implements A, B {
    // Missing implementation
}
```

why java supports call by value?

call by value & call by reference, what java supports?

two interfaces with same methods with different data types

deep copy and shallow copy code

static keyword reference?

string vs string builder vs string buffer

method reference in java

method binding in java

final method real world example

```
class Dog {  
  
    private static int a = 10;  
    private int b;  
  
    public Dog(int b){  
        this.b = b;  
    }  
  
    public static int getA() {  
        return a;  
    }  
  
    public int getB() {  
        return b;  
    }  
  
    I  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
        Dog dog = new Dog(100);  
        dog.getB();  
        Dog.getA();  
    }  
  
}
```

```
protected interface Test {  
}
```

can we do this?

interface access modifiers?

```
package com.prep;  
  
interface speakable {  
    public void speak();  
}  
  
abstract class Animal implements speakable {  
  
}  
  
class Dog extends Animal {  
  
    @Override  
    public void speak() {  
        System.out.println("Bark");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
        Animal dog = new Dog();  
        dog.speak();  
    }  
}
```

interface methods

```

class Student {

    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public String getName() {
        return name;
    }
}

```

so `this` is also a immutable?

how can we make a java object immutable?

Error vs Exception

Explain methods vs functions

```

public class Test {
    public static void main (String[] args) {
        HashSet<Student> set = new HashSet<> ();
        set.add (new Student (1)); set.add(new Student (1));
        System.out.println(set.size ());
    }
}

```

give only output

```

import java.util.*;

public class Test {

    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();

        list.add(1);
        list.add(2);
        list.add(3);

        Iterator<Integer> it = list.iterator();

        while(it.hasNext()) {

            int x = it.next();

            if(x == 2)
                it.remove();
        }

        System.out.println(list);
    }
}

```

```

import java.util.*;

public class Test {

    public static void main(String[] args) {

        List<Integer> list = new ArrayList<>();

        list.add(1);
        list.add(2);
        list.add(3);

        for(Integer i : list) {

            if(i == 2)
                list.remove(i);

        }

    }
}

```

```

import java.util.*;

public class Test {

    public static void main(String[] args) {

        Queue<Integer> q = new LinkedList<>();

        q.add(10);
        q.add(20);
        q.add(30);

        System.out.println(q.poll());
        System.out.println(q.peek());

    }

}

```

```

import java.util.*;

public class Test {

    public static void main(String[] args) {

        HashMap<Integer,String> map = new HashMap<>();

        map.put(1,"A");
        map.put(1,"B");

        System.out.println(map);
    }
}

import java.util.*;

public class Test {
    public static void main(String[] args) {

        HashSet<Integer> set = new HashSet<>();

        set.add(10);
        set.add(20);
        set.add(10);

        System.out.println(set);
    }
}

```

is hashmap thread safe or not?

tree set vs hash set which performance is high

can we have more than one null values in hashmap

can hash set have key value pair?

is hashset and hashmap contains null?

scenerio where we use linkedlist instead of arraylist

in which random access is fast arraylist or array

why we can't use premetive datatypes in generics?

what is the use of oops

what are the different ways to print in java

best way to take input in java?

best between bufferreader and scanner?

then why we use scanner usually if bufferreader is faster

explain stack unwinding

```
package exceptionHandling;

public class temp {
    static void main() {
        try{
            throw new ArithmeticException("exception");
        } finally{
            System.out.println("finally block");
        }
    }
}
```

Output:

finally block

Exception in thread "main" java.lang.ArithmeticException: exc
ption

at exceptionHandling.temp.main(temp.java:6)

explain behind the scene

JAVA EXCEPTION FLOW – WHAT HAPPENS BEHIND THE SCENES

Code

```
package exceptionHandling;

public class temp {
    static void main() {
```

```
try{
    throw new ArithmeticException("exception");
} finally{
    System.out.println("finally block");
}
}
```

Observed Output

finally block

Exception in thread "main" java.lang.ArithmeticException: exception
at exceptionHandling.temp.main(temp.java:6)

1. Important Observation

There is:

- a try block
- a finally block
- NO catch block

The exception is thrown but not handled.

2. Step-by-Step Execution

Step 1 – JVM calls the main method

JVM starts executing:

temp.main()

Step 2 – Execution enters the try block

```
try{  
    throw new ArithmeticException("exception");  
}
```

The statement:

```
throw new ArithmeticException("exception");
```

creates an exception object and throws it immediately.

At this moment:

- normal program flow stops
- JVM starts searching for a catch block.

Step 3 – JVM checks for a catch block

The structure is:

```
try { }  
finally { }
```

There is NO catch block.

So the exception is still unhandled.

Before propagating the exception further, Java must execute the finally block.

Step 4 – finally block execution

Java guarantees that the finally block executes regardless of:

- exception thrown
- return statement
- normal execution

So the JVM executes:

```
finally{  
    System.out.println("finally block");  
}
```

Output:

finally block

Step 5 – Exception Propagation

After the finally block finishes, the exception is still not handled.

So JVM propagates it up the call stack.

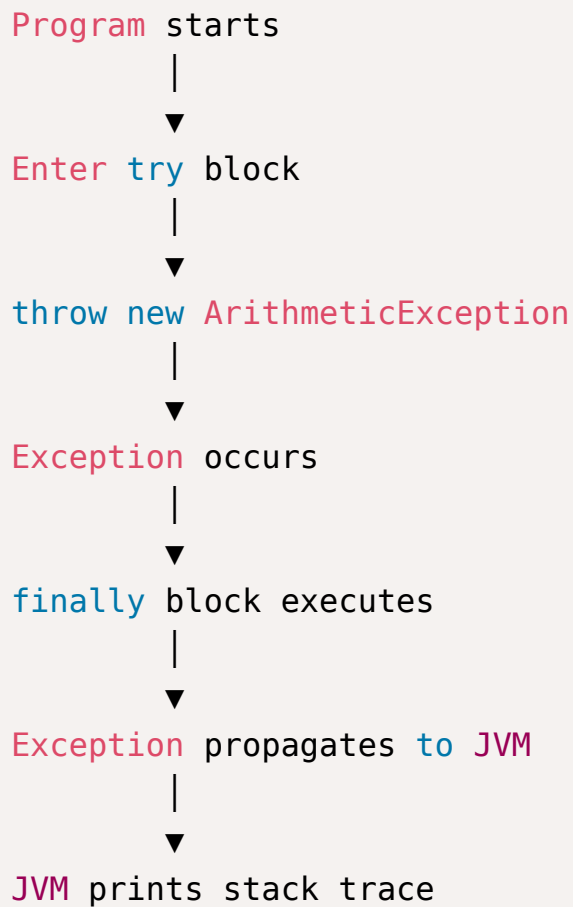
Because there is no caller handling it, the JVM's default exception handler prints the exception.

3. Why This Output Order Happens

Order of events:

1. `Exception` thrown
 2. `finally` block executes
 3. `JVM` prints unhandled exception
-

4. Execution Timeline



5. Stack Trace Explanation

`Exception` in thread `"main"`
`java.lang.ArithmeticException: exception`

Meaning:

An `ArithmeticException` occurred in the main thread.

at exceptionHandling.temp.main(temp.java:6)

Meaning:

The exception happened in:

- class → temp
- method → main
- line → 6

6. What If There Was a Catch Block?

Example

```
try{
    throw new ArithmeticException("exception");
}
catch(ArithmeticException e){
    System.out.println("caught");
}
finally{
    System.out.println("finally block");
}
```

Output

```
caught
finally block
```

Now the exception is handled, so `JVM` does not print the stack trace.

7. Important Rule

`finally` block always executes:

- ✓ when exception occurs
- ✓ when exception is handled
- ✓ when exception is not handled

EXCEPT in rare cases like:

- `System.exit()`
 - JVM crash
-

8. Extra Note (Important)

Your method is written as:

```
static void main()
```

But JVM entry point should be:

```
public static void main(String[] args)
```

Your code likely runs because it is called manually or from another method.

9. One-Line Interview Explanation

When an exception is thrown inside a `try` block without a `catch` block, Java first executes the `finally` block and then propagates the exception to the JVM, which prints the stack trace.

when compile time and run time be achieved anachived

if we have concret methods in abstract class than

can we create object of abstract class

can we have multiple methods with same name but different return type

we have a parent and child class both have void display method but parent is public and child is protected

```
class Test {  
  
    static {  
        System.out.println("Static block");  
    }  
  
    {  
        System.out.println("Instance block");  
    }  
  
    Test() {  
        System.out.println("Constructor");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
        sout("main")  
  
        System.out.println("First object creation");  
        TEst two = new Test();  
  
        System.out.println("Second object creation");  
        Test one = new Test();  
    }  
}
```

```
}
```

PROGRAM EXECUTION & OUTPUT

Given Code (corrected for syntax)

```
class Test {  
  
    static {  
        System.out.println("Static block");  
    }  
  
    {  
        System.out.println("Instance block");  
    }  
  
    Test() {  
        System.out.println("Constructor");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        System.out.println("main");  
  
        System.out.println("First object creation");  
        Test two = new Test();  
  
        System.out.println("Second object creation");  
        Test one = new Test();  
    }  
}
```

1. Execution Order Rules

Java follows `this` order:

- 1 `Static Block` → when `class` loads (only once)
 - 2 `main()` method starts
 - 3 `Instance Block` → when object created
 - 4 `Constructor` → after instance block
-

2. Step-by-Step Execution

STEP 1 – Program starts

JVM loads the `Main class` and starts the `main()` method.

Output:

main

STEP 2 – First object creation message

Output:

First object creation

STEP 3 – Object creation

`Test two = new Test();`

When `class Test` is used for the first time:

1. **Static** block runs (once)
2. **Instance** block runs
3. **Constructor** runs

Output:

Static block

Instance block

Constructor

STEP 4 – **Second** object creation message

Output:

Second object creation

STEP 5 – **Second** object creation

Test one = **new** **Test**();

Static block will **NOT** run again because the **class** is already loaded.

Only:

Instance block

Constructor

Output:

Instance block

Constructor

3. Final Output

main

First object creation

Static block

Instance block

Constructor

Second object creation

Instance block

Constructor

4. Important Concept

Block Type

Runs How Many Times

Static Block Only once when class loads

Instance Block Every time object is created

Constructor Every time object is created

Objects created: 2

Instance Block → 2 times

Constructor → 2 times

Static Block → 1 time

OUTPUT ANALYSIS (Based on the code shown in the image)

Assumed Structure From the Image

Class: Test

Contains:

- Static Block
- Instance Block

- Constructor

Main Class creates two objects and prints a message between them.

1. Test Class (Reconstructed)

```
class Test {  
  
    static {  
        System.out.println("Static Block");  
    }  
  
    {  
        System.out.println("Instance Block");  
    }  
  
    Test() {  
        System.out.println("Constructor");  
    }  
}
```

2. Main Class

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Test t1 = new Test();  
  
        System.out.println("Second object creation");  
    }  
}
```

```
        new Test();  
    }  
}
```

3. Execution Step-by-Step

STEP 1 – Class Loading

When the class `Test` is used for the first time:

Static Block executes (only once)

Output:

Static Block

STEP 2 – First Object Creation

```
Test t1 = new Test();
```

Execution order:

Instance Block

Constructor

Output:

Instance Block

Constructor

STEP 3 – Print Statement

```
System.out.println("Second object creation");
```

Output:

Second object creation

STEP 4 – Second Object Creation

```
new Test();
```

Execution order again:

Instance Block

Constructor

Output:

Instance Block

Constructor

4. Final Output

Static Block

Instance Block

Constructor

Second object creation

Instance Block

Constructor

5. Key Rule

Execution Order:

Static Block → runs once when class loads

Instance Block → runs for every object
Constructor → runs for every object

if we create 2 objects in other class what will be the output

OUTPUT WHEN 2 OBJECTS ARE CREATED IN ANOTHER CLASS
(Static Block + Instance Block + Constructor)

1. Example Program

```
class Demo {  
  
    static {  
        System.out.println("Static Block");  
    }  
  
    {  
        System.out.println("Instance Block");  
    }  
  
    Demo() {  
        System.out.println("Constructor");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        Demo d1 = new Demo();  
        Demo d2 = new Demo();  
    }  
}
```

```
}  
}
```

2. Execution Process

STEP 1 – Class Loading

When the program runs and the `class Demo` is first used, the `JVM` loads the `class` into memory.

→ `Static Block` executes only once.

STEP 2 – First Object Creation

```
Demo d1 = new Demo();
```

Execution order:

1. Instance Block
2. Constructor

STEP 3 – Second Object Creation

```
Demo d2 = new Demo();
```

Execution order again:

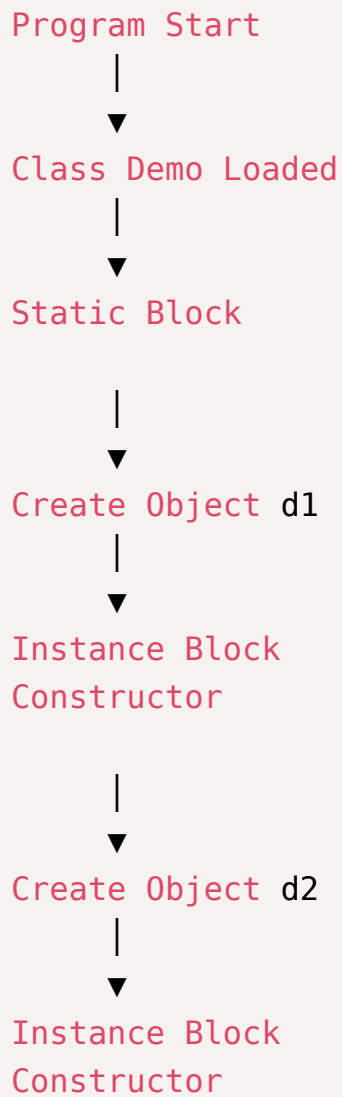
1. Instance Block
 2. Constructor
-

3. Final Output

Static Block
Instance Block

Constructor
Instance Block
Constructor

4. Visualization



5. Important Points

Feature	Behavior
Static Block	Runs only once when <code>class</code> loads
Instance Block	Runs <code>for</code> every object creation
Constructor	Runs <code>for</code> every object creation

Objects Created	Instance Block Runs
1 Object	1 time
2 Objects	2 times
3 Objects	3 times

6. Short Interview Answer

If two objects are created:

Static Block → runs once

Instance Block → runs twice

Constructor → runs twice

if we have static and instance block both. then

instance block

explain open closed protocol

explain singleton design pattern